



HLS: A High-Performance Local Search Method for Qubit Mapping Problem

Hongyuan ZHANG¹, Shenghua CAO¹, Xiaoran ZHAO¹✉, Chuan LUO¹

1. School of Software, Beihang University, Beijing 100191, China

Received month dd, yyyy; accepted month dd, yyyy

E-mail: zhaoxiaoran@buaa.edu.cn.

© Higher Education Press 2026

Abstract

Currently, the development of quantum computing is in the noisy intermediate-scale quantum (NISQ) stage. In the NISQ era, subject to connectivity constraints, quantum circuits require a specific compilation process before being executed on the NISQ devices. Qubit mapping plays a critical role in quantum circuit compilation, where auxiliary gates such as SWAP gates and bridges are inserted to satisfy connectivity constraints. These additional gates significantly increase circuit depth and reduce fidelity. Consequently, minimizing the number of auxiliary gates becomes essential for enhancing circuit reliability, giving rise to the qubit mapping problem (QMP). To address this challenge, we propose HLS, a high-performance local search method for solving QMP. HLS introduces a bridge-balanced scoring function to reduce the evaluation bias of bridge operations that have plagued previous QMP algorithms. Furthermore, HLS utilizes a potential-guided tie-breaking rule, which markedly reduces the probability of ties during operation evaluation, leading to more deterministic operation selection. Last, HLS iteratively optimizes random mappings using SWAP operations, improving the quality of the initial mapping. Extensive experiments on 5 NISQ devices and 4 circuit sets demonstrate that HLS can generate circuits significantly smaller than current state-of-the-art algorithms, indicating HLS's effectiveness in solving QMP.

Key words

Noisy intermediate-scale quantum (NISQ), qubit mapping, local search

1 Introduction

Quantum computing, a computational paradigm governed by the principles of quantum mechanics and utilizing quantum bits (qubits) as its fundamental units [1], has attracted significant attention due to its potential to surpass classical computing capabilities in specific application domains such as quantum chemistry simulation [2], cryptography [3], and combinatorial optimization [4].

To implement quantum algorithms in practice, they are commonly described using the quantum circuit. A quantum circuit represents computations as sequences of quantum gates applied to qubits, including single-qubit gates, two-qubit gates, etc.

In reality, however, quantum devices are characterized by low operational fidelity and substantial physical limitations in the current noisy intermediate-scale quantum (NISQ) era [5, 6]. A critical constraint is the limited connectivity between physical qubits, which restricts the execution of two-qubit gates (such as the CNOT gate) to pairs of qubits that are directly adjacent in the device's architecture.

To execute arbitrary quantum circuits on such restricted devices, a quantum compilation process is required to translate the logical algorithm into executable instructions. This process relies critically on qubit mapping, a transformation that adapts the abstract logical circuit to a physical one in order to satisfy the connectivity constraints

of NISQ devices. The transformation begins with an initial mapping, which assigns each logical qubit to a specific physical qubit on the device. During the transformation, auxiliary gates (including SWAP gates and bridges) are inserted to ensure the interaction between qubits is directly executable. However, each additional gate introduces noise and reduces overall circuit fidelity, which makes improving the quality of quantum mapping the central challenge in solving the qubit mapping problem (QMP). The optimal QMP of minimizing circuit size has been proven NP-Complete [7, 8], and the development of efficient and high-quality heuristic algorithms has become a major research focus.

Existing approaches to QMP can be broadly categorized into two classes: exact methods based on constraint encoding and heuristic search methods. Exact methods formulate QMP as a mathematical programming problem (e.g., Integer Linear Programming [9] or Boolean Satisfiability [10, 11]), and leverage specialized solvers to find optimal solutions. While capable of finding optimal mappings for small-scale circuits, these methods struggle to scale to larger problems due to the exponential computational overhead inherently associated with the NP-Complete nature of the QMP [7, 8] and insufficient tuning for this specific problem.

For practical quantum compilation, heuristic search methods are predominant due to their favorable scalability and efficiency. In 2018,

Zulehner et al. [12] proposed the first heuristic for NISQ-era QMP. Their critical innovation was the strategy of partitioning the logical circuit into layers, enabling more dynamic and progressive mapping. Among the heuristic search methods, the SABRE algorithm [13] is a landmark contribution. It employs a bidirectional search strategy with a decayed cost function, effectively balancing circuit depth and gate count to produce high-quality approximate solutions for large circuits. Subsequent research has primarily focused on enhancing three key aspects of the SABRE framework: 1) improving the initial mapping, using techniques based on subgraph isomorphism [14–16], simulated annealing [17], or reinforcement learning [18]; 2) optimizing the search strategy itself by incorporating A* search [18, 19], adaptive divide and conquer [14], or tabu search [15]; and 3) some advanced methods [20–22] introduce the bridge operation to achieve connectivity more efficiently.

Within the heuristic paradigm, local search algorithms are widely recognized for their flexibility and practical efficacy. These methods iteratively refine an initial solution by exploring and transitioning to superior solutions within a defined neighborhood [23]. In recent years, EffectiveQM (EQM) [22] is a state-of-the-art local search QMP algorithm that achieves an effective trade-off between solution quality and runtime. It introduces a mode-aware search strategy to mitigate the limitations of conventional local search and an advanced potential-guided scoring function to quantify the actual benefit of an operation. However, current QMP algorithms exhibit several shortcomings. First, for the scoring functions used in QMP algorithms that introduced the bridge operation [20–22], the distinction between the characteristics of SWAP and bridge operations could be further emphasized. In certain circuits, this may lead to a preference for SWAP over bridge operation when the latter is more favorable. Second, the scoring mechanisms of existing local search QMP algorithms [21, 22] lack fine-grained discrimination. This leads to frequent score ties, empirically observed in approximately 24% of decision steps, where random choices are forced, and solution quality is consequently degraded. Lastly, although some research, including EQM, refines the initial mapping via reverse traversal, this approach still relies on a randomly generated mapping as the starting point, which constrains the overall effectiveness of the search process.

To directly address these limitations and further advance heuristic QMP algorithms, this paper proposes a novel high-performance local-search method for QMP, which we refer to as HLS. Specifically, our approach tackles the aforementioned shortcomings through three key innovations: Firstly, we noticed that the bridge operations in the QMP process can only function under specific circuit states. Previous QMP algorithms did not explicitly incorporate this characteristic of bridge, which could cause the search to overlook moves that make the circuit bridge-favorable, thereby reducing the chances of the bridge and leading to a decrease in performance. To this end, we introduce penalties and rewards to encourage the selection of operations that make the circuit state more suitable for the bridge, thereby increasing the likelihood of bridge operations being selected and reducing suboptimal selection. Secondly, to resolve the issue of frequent score ties, we propose an

advanced tie-breaking rule inspired by the potential-guided strategy of EQM. Instead of resorting to random selection, we treat a tie as a critical decision point requiring optimized resolution. When multiple operations receive identical primary scores, HLS executes a secondary, deeper lookahead as a refined criterion. While incurring only a negligible time overhead, this strategy eliminates random choices and leads to significantly better decision-making. And thirdly, to mitigate the dependency on a purely random initial mapping, we introduce a heuristic pre-processing phase before executing any logical gates. This phase iteratively adjusts the mapping by applying a series of chosen SWAP operations, thereby providing a locally optimized, stable, and high-quality starting configuration for the main search algorithm.

To evaluate HLS, we conduct extensive experiments on 4 logical circuit sets across 5 distinct NISQ devices. Results demonstrate that HLS consistently inserts fewer auxiliary CNOT gates compared to state-of-the-art QMP algorithms, showing its superior capability in circuit size reduction and robustness.

The major contributions of this paper are summarized as follows:

- We propose a novel high-performance local search method HLS, which exceeds the state-of-the-art QMP algorithms in solution quality.
- We design three core mechanisms that constitute HLS: a bridge-balanced scoring function, a potential-guided tie-breaking rule, and an iterative initial mapping adjustment strategy, effectively resolving the limitations of previous methods.
- We conduct comprehensive experiments on benchmark circuit sets. The results not only demonstrate the overall superiority of HLS, but also validate the effectiveness of each proposed mechanism through detailed ablation studies.

The remainder of this article is organized as follows. Section 2 provides the necessary background on the Qubit Mapping Problem and the general framework of a local search QMP Algorithm. Section 3 details our proposed improvements. A comprehensive experimental evaluation, comparing HLS against state-of-the-art methods, is presented in Section 4. Finally, Section 5 concludes the paper.

■ 2 Background

In this section, we introduce the fundamental concepts required to understand the Qubit Mapping Problem (QMP) and our proposed algorithm. We first define the logical input (quantum circuits) and the physical target (NISQ devices), followed by a formal formulation of QMP and a brief introduction of the general local search QMP algorithm framework.

2.1 Quantum Circuit

The quantum circuit is a model used to describe quantum algorithms. Analogous to classical circuits, which process 0/1 bits through operations such as AND, OR, and NOT gates, quantum circuits process the interactions of qubits by quantum gates [24]. Single-qubit gates and the CNOT gate, among two-qubit gates, are referred to as elementary gates [25]; non-elementary gates can be converted to combinations of

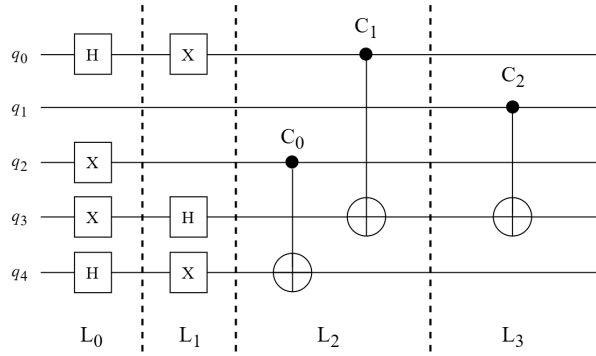


Fig. 1 Example of a logical circuit.

elementary gates [26]. Additionally, a complete quantum circuit includes the readout of results, where measurement collapses the quantum state to yield a classical bit. Because the measurements are fixed in the logical circuit and are independent of the qubit mapping, recent studies [16, 21, 22] concentrate on circuits with elementary gates, so in this work, we focus solely on quantum circuits composed of elementary gates and do not consider the measurement.

Fig. 1 shows an example of a quantum circuit only composed of elementary gates, involving 5 qubits (denoted as q_0 to q_4), 7 single-qubit gates (including H and X gates), and 3 CNOT gates (denoted as C_0 to C_2). This circuit is executed from left to right. For qubit q_1 , the CNOT gate C_2 is the first gate to be executed. However, for the other qubit q_3 involved in C_2 , this gate is the fourth one to be executed. Therefore, for q_1 , the execution of C_2 must wait until the corresponding operation on q_3 is ready. This leads us to consider temporal partitioning of the circuit. Based on dependencies among gates, we divide quantum gates that can be executed simultaneously into the same layer. As a result, this example is divided into 4 layers, from L_0 to L_3 . For the gates in the leftmost layer L_0 , they have no data dependencies and can be executed immediately.

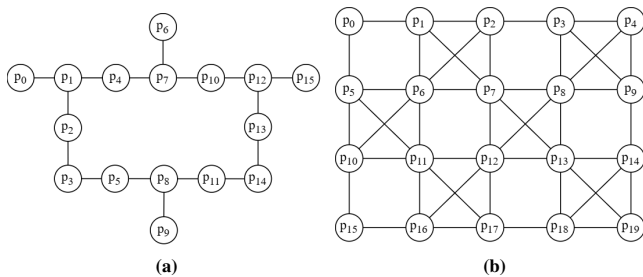


Fig. 2 CGs of two NISQ devices. (a) CG of IBM Guadalupe; (b) CG of IBM Tokyo.

2.2 NISQ Device

In the current era, quantum circuits are executed on NISQ devices. Research institutions and technology companies such as IBM and Google have released their NISQ devices. Their distinct approaches have shown quantum computers' potential in fields like healthcare, electronics, and life sciences [27, 28].

These devices exhibit constraints in the layout and connectivity

of physical qubits, which can be represented by a coupling graph $CG(V, E)$. Here, V denotes the set of physical qubits, and E is the set of undirected edges. An edge $(u, v) \in E$ indicates that a CNOT gate $C(u, v)$ or $C(v, u)$ can be directly performed between qubits u and v , conforming to the physical constraints of the device. Single-qubit gates (such as H or X gates) are not subject to connectivity constraints and can be directly applied, so in the mapping process, we only consider the CNOT gates.

Fig. 2 illustrates the coupling graphs of two typical NISQ devices: (a) the IBM Guadalupe device, containing 16 qubits and 16 edges, and (b) the IBM Tokyo device, comprising 20 qubits and 43 edges. This 2 devices and the other 3 devices with more qubits and edges are used in the experiment section.

2.3 Qubit Mapping Problem

2.3.1 Qubit Mapping

The process of executing a quantum circuit on an NISQ device involves assigning logical qubits to physical qubits and inserting auxiliary gates. Typically, the target quantum circuit is referred to as the logical circuit, while the circuit actually executed on the device is called the physical circuit. The transformation from a logical circuit to a physical circuit is known as qubit mapping. A valid qubit mapping must satisfy the following two conditions: a) all quantum gates can be executed sequentially under the device's physical constraints; b) the physical circuit must be functionally equivalent to the logical circuit.

The process first requires assigning each logical qubit to a physical qubit, represented by an injective mapping function π . While device constraints are inherently satisfied for single-qubit gates, for two-qubit gates such as a CNOT gate $C(q_i, q_j)$, the shortest distance between the two mapped physical qubits $\pi(q_i)$ and $\pi(q_j)$ must be 1 in the device's coupling graph, as shown in Eq. (1). For convenience, we use the term distance- x CNOT to refer to a CNOT gate for which the shortest distance between its associated qubit pair is x .

$$\text{shortest_dist}(\pi(q_i), \pi(q_j)) = 1. \quad (1)$$

Under the current mapping, if the two logical qubits associated with a CNOT gate operation are mapped to physical qubits with a distance greater than 1, this CNOT gate cannot be directly executed. In such cases, auxiliary quantum gates must be introduced to satisfy the device's connectivity constraints. Existing research primarily uses two operations to address this issue: the SWAP operation and the bridge operation.

2.3.2 SWAP operation

SWAP operation exchanges the states of two adjacent qubits, and is implemented using three CNOT gates; it is equivalent to swapping the mapping values of the two qubits. For a CNOT gate $C(q_i, q_j)$ with $\text{dist}(\pi(q_i), \pi(q_j)) > 1$, we can find a q_k satisfying $\text{dist}(\pi(q_i), \pi(q_k)) = 1$, and do $\text{SWAP}(q_i, q_k)$. Then the physical distance between qubits q_i and q_j becomes the original shortest distance between q_k and q_j . Since $\text{dist}(\pi(q_i), \pi(q_k)) = 1$, this distance change is bounded by 1. Also, the distances between q_k and other qubits are updated.

As shown in Fig. 3, the original mapping was $\pi(q_1) = p_1$ and $\pi(q_2) = p_2$. If the distance between p_1 and p_2 is 1 in the device, the

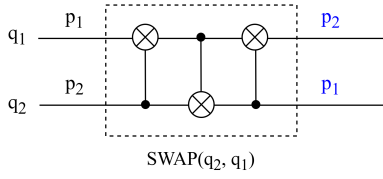


Fig. 3 Example of a SWAP operation.

SWAP operation can be performed. It is composed of three CNOTs. After $SWAP(q_2, q_1)$, the mapping changes to $\pi(q_1) = p_2, \pi(q_2) = p_1$.

By strategically inserting SWAP operations, the distances between qubits can be gradually adjusted, ultimately reducing the distance between a target pair to 1 to satisfy device constraints for CNOT.

2.3.3 Bridge operation

Bridge operation can enable CNOT gates between qubits at any distance. Previous research [20–22] mainly studied the use of bridge operation for distance-2 CNOT gates. Specifically, for a distance-2 CNOT gate $C(q_i, q_j)$, assuming an intermediate node q_v lies on the shortest path between q_i and q_j , the operation introduces 4 CNOT gates which collectively emulate the effect of $C(q_i, q_j)$.

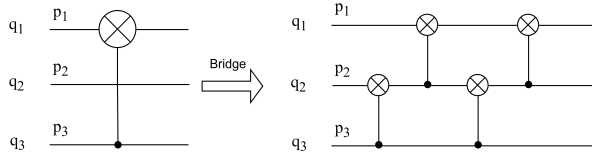


Fig. 4 Example of a bridge operation.

As shown in Fig. 4, under the mapping $\pi(q_1) = p_1, \pi(q_2) = p_2$, and $\pi(q_3) = p_3$. If the distance between p_1 and p_2 , p_2 and p_3 is 1 in the device, bridge operation can be performed. This operation replaces a distance-2 CNOT gate with a sequence of 4 distance-1 CNOT gates, thereby satisfying the device's connectivity constraints. Unlike SWAP operations, the bridge operation does not alter the mapping function π . In terms of gate overhead, it introduces three additional CNOT gates to enable a distance-2 CNOT gate.

2.3.4 Qubit Mapping Problem Formulation

Given a quantum logical circuit $LC(Q, G)$, where Q is the set of logical qubits and G is the set of elementary gates, and a coupling graph $CG(V, E)$ of a NISQ device satisfying $|V| \geq |Q|$, the qubit mapping problem requires finding a mapping function π assigning logical qubits to physical qubits and generating a physical circuit $PC(Q_p, G_p)$. Here, $Q_p \subseteq V$ represents the occupied physical qubits, and G_p includes both the original logical gates and any auxiliary gates (e.g., CNOT gates introduced by SWAP or bridge operations) inserted to satisfy the device's connectivity constraints. The physical circuit must fulfill the constraints of the device and functional equivalence mentioned above.

Since inserting auxiliary gates increases circuit depth and introduces additional noise, it is essential to minimize the number of auxiliary CNOT gates, specifically, to optimize the use of SWAP and bridge operations while ensuring a valid mapping. This study focuses

Algorithm 1: Function Local_Search_Qubit_Mapping

Input: $LC(Q, G)$: logical circuit;
 $CG(V, E)$: NISQ device coupling graph;
Output: PC : physical circuit;
 π_p : initial mapping of PC ;

```

1  if  $|V| < |Q|$  then
2      return null, null;
3   $PC \leftarrow \emptyset$ ;
4   $\pi_p \leftarrow \emptyset$ ;
5  while no terminal signal do
6       $PC' \leftarrow \emptyset$ ;
7       $\pi_0 \leftarrow \text{get\_init\_mapping}()$ ;
8      while no restart signal do
9           $\pi' \leftarrow \pi_0$ ;
10         while mapping is not finished do
11             Check executable gates;
12              $ops \leftarrow$  all feasible operations;
13             Evaluate  $ops$  and select one as  $best\_op$ ;
14             /* check op strategy:
15              if  $best\_op$  is not qualified then
16                  fallback( $PC', \pi'$ );
17             */
18             exec_op( $best\_op, PC', \pi'$ );
19         if  $|PC'| < |PC|$  then
20              $PC \leftarrow PC'$ ;
21              $\pi_p \leftarrow \pi_0$ ;
22          $\pi_0 \leftarrow \text{neighborhood}(\pi')$ ;
23         /* reverse mapping:
24         gate_list  $\leftarrow$  reverse(gate_list);
25          $\pi_0 \leftarrow \pi'$ ;
26         */
27     return  $PC, \pi_p$ ;
```

on minimizing the number of inserted CNOT gates as the primary optimization objective.

2.4 Local Search QMP Algorithm

This section presents the general framework of the local search algorithm for qubit mapping on NISQ devices.

2.4.1 General Framework

We present the algorithm in Algorithm 1. For a QMP instance, as defined in Section 2.3.4, the inputs consist of a logical circuit $LG(Q, G)$ and a NISQ device's coupling graph $CG(V, E)$, the outputs are the physical circuit PC and corresponding initial mapping function π_p .

For QMP, if the number of physical qubits $|V|$ is fewer than the number of logical qubits $|Q|$, this QMP cannot be solved, and the algorithm returns a null value (Lines 1-2 in Algorithm 1).

The solving process starts from an initial mapping π_0 . Since the number of possible π_0 is given by the permutation $P_{|V|}^{|Q|}$, which can easily reach factorial scale, it is usually feasible to only select a subset of initial mappings for heuristic search.

To control the number of search iterations, a terminal signal is introduced, and the condition to terminate should be well considered to balance the runtime and solution quality. PC and π_p are maintained to record the currently optimal physical circuit and corresponding initial mapping, which is compared and updated after each iteration (Lines 15-17 in Algorithm 1).

In a single iteration, an empty circuit PC' is initialized to record the resulting physical circuit. The mapping process starts from an initial mapping π' , π' can be random or constructed, as mentioned in Section 1, many studies have designed specific strategies for this stage. The process first determines whether the current mapping is complete by checking the gates in LC . If a gate satisfies the connectivity constraints, it is directly inserted into PC' ; otherwise, it is logged for subsequent processing. This process repeats until all gates in the leftmost layer are found. If the leftmost layer is empty, it indicates that the mapping is complete, and this iteration ends (Lines 10-11 in Algorithm 1).

All feasible operations involve candidate SWAP and bridge operations associated with gates in the leftmost layer. Specifically, for the qubit pair of a CNOT gate, any third qubit that is at a distance 1 from a qubit in the pair corresponds to a potential SWAP operation. If the distance of the pair is exactly 2, a bridge operation is added. Then ops is evaluated, and the operation with the highest score is selected for execution. In the case of a tie (multiple operations have the same highest score), one of them is selected randomly (Lines 12-13 in Algorithm 1).

The execution of $best_op$ is exactly the insertion of corresponding CNOT gates into PC' . And if $best_op$ is a SWAP operation, π_0 is also updated. However, in some extreme cases, the score of $best_op$ may receive non-positive scores, thereby preventing the mapping progress and leading to an infinite loop. A robust heuristic search algorithm incorporates a fallback mechanism to handle such situations (block comment between lines 13 and 14 in Algorithm 1).

Compared to conventional heuristic search, a local search method selects a suitable neighborhood to iteratively refine the current solution. Both the ILS [21] and EQM [22] algorithms fully take advantage of reverse mapping. Specifically, reverse mapping uses the result of a previous mapping as the initial mapping and starts the next search on the reversed circuit. Since the mapping result inherently contains global information about the entire circuit, it is a high-quality neighborhood. The local search method using reverse mapping is shown in block comment between lines 18 and 19 in Algorithm 1.

2.4.2 Scoring Function

The scoring function in QMP is used to evaluate the possible SWAP and bridge operations, and it corresponds to the evaluation of ops in line 13 in Algorithm 1. In a compilation process aimed at making all CNOT gates in the logical circuit satisfy connectivity constraints, the most direct benefit comes from the extent to which the operation brings a CNOT gate closer to satisfying the constraints. This can be expressed by the change in distance, as shown in Eq. (2), where g denotes a CNOT gate, π is the current mapping, op is a SWAP operation, $d(g, \pi)$ represents the distance between the associated qubit

pair under π , and $\pi \triangleleft op$ denotes the mapping after executing op . For bridge operations, which do not change the mapping, the effect of enabling the execution of a distance-2 CNOT is equivalent to reducing the distance by 1. Therefore, this term is set to 1.

$$\zeta(g, \pi, op) = d(g, \pi) - d(g, \pi \triangleleft op) \quad (2)$$

ILS involves the impact of whether the operation can make a CNOT gate directly executable into its scoring function. Since it places greater emphasis on advancing the mapping process, it largely avoids the blocking situations mentioned in Section 2.4, thereby improving search efficiency. This scoring function is denoted as distance score (ds), which better quantifies the direct effect of an operation op under the mapping π .

$$ds(op, \pi) = \sum_{g \in G_\varphi} \zeta(g, \pi, op) + \Psi \quad (3)$$

Eq. (3) combines the total changes in physical distance for CNOT gates in the front φ layers and the number of logical gates that become executable after applying op (denoted as Ψ).

EQM introduces potential benefits on the basis of ds , which is called potential-guided scoring function (ps). As shown in Eq. (4), ps involves the impact of the current operation on subsequent operations by adding a term representing the maximum benefit from subsequent operation op' after the execution of op . If the mapping is complete after the execution of op , op is simply chosen.

$$ps(op, \pi) = ds(op, \pi) + \max\{ds(op', \pi \triangleleft op)\} \quad (4)$$

ps effectively integrates both immediate and potential benefits when evaluating a given operation. It is a forward-looking strategy and helps prevent premature convergence to local optima.

3 Overview

The overall framework of HLS is consistent with the content in Section 2.4, and it uses the same terminal signal and restart signal as EQM [22] and reverse mapping. The core mechanisms of HLS are introduced in the heuristic search scoring stage, the decision stage, and the initial mapping stage. Specifically, the first two mechanisms correspond to line 13 in Algorithm 1, and the last mechanism corresponds to the function `get_init_mapping` in Algorithm 1 line 7. This section details these core mechanisms in HLS.

3.1 Bridge-balanced Scoring Function

We introduced the scoring function of the local search QMP algorithm in Section 2.4, both ds and ps perform well in evaluation. However, by simply equating bridge operations to a distance reduction of 1, these scoring functions do not fully consider the inherent characteristics of bridge operations, which can result in suboptimal evaluation outcomes.

The consideration of potential benefits is well-founded and can essentially be viewed as an extension of ds , so we focus our analysis on ds , as shown in Eq. (3). For the distance-change part, SWAP operations alter the mapping function π and thus contribute to this term. In contrast, bridge operations preserve π and receive a value of 1 for its functional equivalence. As for the executable gates part,

both SWAP and bridge operations are considered equally, as each can potentially enable the execution of subsequent gates.

Regarding the differences in the impact of distance changes when ds remains the same, the underlying reason is that some physical qubits are not equivalent [29]; their positions in CG inherently possess different values, which are difficult to quantify directly. One intuitive factor is node degree; we explored incorporating node degree into the scoring function. For instance, encouraging SWAP operations toward higher-degree qubits to increase the optionality of subsequent decisions or toward lower-degree qubits to accelerate the process. However, experimental results indicate that the effectiveness of such strategies is highly dataset-dependent, and no universal improvement was observed. Therefore, while intuitively meaningful, explicit degree-based adjustments lack generalizability and were not adopted in our generalized scoring function.

As introduced in Section 2, the bridge operation specifically targets distance-2 CNOT gates, using an intermediate qubit to indirectly implement this gate. This inspires us to assign an additional score to operations that adjust gate distance to exactly 2, whether by reducing distance from 3 to 2 or increasing from 1 to 2. Such distance changes add the possibility of using bridge operations, thereby enriching future operational choices.

It is worth emphasizing that, unlike strategies such as moving qubits to higher-degree nodes, which tend to favor specific structural properties of CG , the opportunity introduced by bridge operations does not inherently prefer particular qubit positions. This makes it a more general strategy with broader applicability across different NISQ devices and circuit sets.

Correspondingly, while it is beneficial to assign positive scores for distance adjustments toward 2, it is also necessary to consider penalties for operations that increase distance away from 2. For instance, an increase from distance 2 to 3 reduces the opportunity for using bridge operations and should therefore receive a negative score. We do not penalize distance reduction from 2 to 1 even though it also eliminates the chance for bridge operations, because allowing immediate gate execution is essential to move the mapping forward, and penalizing such a step would hinder the process. We still reward an increase from 1 to 2 since that scenario only occurs in deeper layers and does not block current execution.

For a given operation op , we define a bridge bonus (brib) as follows:

$$brib = \begin{cases} 1, & \text{if } od = 1 \text{ or } 3, \text{ and } sd = 2 \\ -1, & \text{if } od = 2 \text{ and } sd = 3 \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

There, od denotes the original physical distance between two qubits, and sd denotes the distance after applying op . Since only SWAP operations alter the qubit distance, for bridge operations, $brib = 0$.

The $brib$ is incorporated into ds in Eq. (3). To integrate this new term while preserving the original evaluation of operations, considering that this rating is only applicable when considering bridge operations, while the original ds applies to both SWAP and bridge

Table 1 Bridge operations per decision for RevLib circuit set on 5 devices

Alg.	TKY	SCM	GDL	RCT	TRN	average
EQM	10.03	8.29	4.98	4.62	4.46	6.48
EQM*	11.11	9.48	5.54	5.12	4.94	7.24

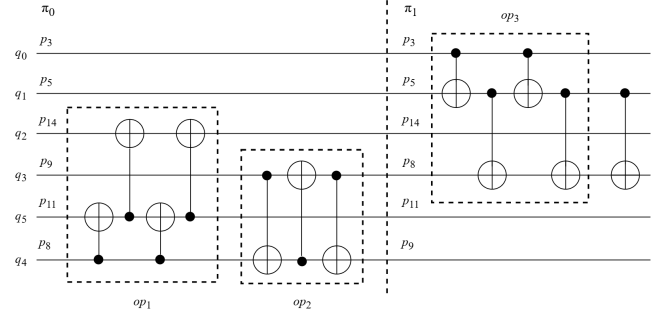


Fig. 5 Generated physical circuit (omit H and X gates).

operations, we assign a weight ratio of 1:2 between the $brib$ and ds . This leads to the following bridge-balanced base score (brs) formula:

$$brs(op, \pi) = 2ds(op, \pi) + \sum_{g \in G_\varphi} brib \quad (6)$$

The overall form of our bridge-balanced scoring function (brs) is the same as ps , with only brs replacing ds .

$$brs(op, \pi) = brs(op, \pi) + \max\{brs(op', \pi \triangleleft op)\} \quad (7)$$

Example 1. Given a logical circuit as shown in Fig. 1 and a NISQ device coupling graph as shown in Fig. 2a, we set the parameter $\varphi = 2$. Under an initial mapping π : $\pi(q_0) = p_0$, $\pi(q_1) = p_1$, $\pi(q_2) = p_2$, $\pi(q_3) = p_4$ and $\pi(q_4) = p_3$, the distance between q_2 and q_4 is 1, $C_0(q_4, q_2)$ can be executed directly, while $C_1(q_0, q_3)$ is not due to insufficient connectivity. Although the distance between q_1 and q_3 is 1, $C_2(q_1, q_3)$ still has to wait the execution of $C_1(q_0, q_3)$. To resolve this, we evaluate the permissible operations (op). Here, the SWAP operation $S(p_0, p_1)$ changes the distance of $C_1(q_0, q_3)$ by 1, to the subsequent layer $C_2(q_1, q_3)$ by -1 . The total change is $1 + (-1) = 0$. After the execution of $S(p_0, p_1)$, $C_1(q_0, q_3)$ is immediately executable ($\Psi = 1$). The distance of $C_2(q_0, q_3)$ increases from 1 to 2, giving a bridge bonus of 1, resulting in a total $brib = 1$, then $brs = 2 \times (1 + 1) + 1 = 5$, the maximum benefit from op' (i.e., $SWAP(p_1, p_4)$) is 4, final $brs = 5 + 4 = 9$. Similarly, Bridge operation $B(p_0, p_4)$ yields $brs = 2 \times (1 + 2) + 0 = 6$, and the mapping is complete after the execution of it, $B(p_0, p_4)$ is chosen and $brs = 6$.

We conducted preliminary tests for the RevLib circuit set on 5 NISQ devices (meanings of denotations of circuit sets and devices can be viewed in Section 4) to measure the ratio of bridge operation selections to the total number of decisions made by EQM and a version of EQM which only changed the scoring function to brs (denoted as EQM*). As shown in Table 1, after implementing our bridge-balanced scoring function, the average ratio is significantly increased by 11.7%. The results confirm that brs indeed increases the likelihood of bridges being incorporated into the decision.

Example 2. Given a logical circuit as shown in Fig. 1 and a NISQ device coupling graph as shown in Fig. 2a, we set the parameter $\varphi = 2$. Under the initial mapping π_0 : $\pi_0(q_0) = p_3$, $\pi_0(q_1) = p_5$, $\pi_0(q_2) = p_{14}$, $\pi_0(q_3) = p_9$ and $\pi_0(q_4) = p_8$. During the qubit mapping process, *brs* evaluated an evaluation score of 9 to the bridge operation $B(q_4, q_2)$ and a score of 8 to the SWAP operation $S(q_3, q_4)$, leading to the selection of $B(q_4, q_2)$ for execution. This decision resulted in the successful completion of the mapping with the insertion of only 9 CNOT gates, as illustrated by the result physical circuit in Fig. 5. In contrast, *ps* assigns an identical score of 4 to both candidate operations, creating a tie. This will lead to a random selection with a 50% probability of choosing $S(q_3, q_4)$, ultimately requiring the insertion of at least 12 CNOT gates to complete the mapping. This case intuitively demonstrates the advantage of *brs* in guiding the search toward more beneficial operations.

3.2 Potential-guided Tie-breaking Rule

As introduced in Section 2.4.2, the selection among candidate operations is guided by a scoring function, and when multiple operations receive the same score, one is chosen at random for execution. An effective scoring strategy should provide meaningful guidance in most cases, while still allowing a controlled degree of randomness. Appropriate randomization helps the search escape local optima; however, excessive reliance on random choices can degrade overall search quality by reducing the directionality and efficiency of the optimization process.

We measured the tie rate, exactly, the proportion of decisions where the highest score is shared by multiple operations of ILS and EQM. Experiments on RevLib circuit set across 5 devices are shown in Table 2, EQM obtained an average tie rate of 24.75% and ILS 23.41%. This high incidence of ties suggests that the scoring function provides inadequate guidance, causing the selection to have a relatively high degree of randomness. This highlights a critical opportunity to improve search efficiency by introducing more fine-grained decision criteria.

Inspired by EQM's *ps* that increased search depth improves decision quality at the cost of computational feasibility. We implement the potential-guided score at a depth of 3 for breaking ties. Specifically, when *brs* meets a tie, we execute all operation pairs (*op* and *op'*) with the highest same score, and then compute the maximum benefit from the subsequent operation. This change is expected to require little time consumption but reach better results.

Under this strategy, the tie-breaking score(*ts*) can be expressed as:

$$ts(op, \pi) = \max\{brs(op'', \pi \triangleleft op')\} \quad (8)$$

We present the complete process of scoring and decision-making in HLS in Algorithm 2, which corresponds to line 13 in Algorithm 1. Some variables such as *PC'* and *CG* are omitted. This enhanced operation selection process begins with setting the initial values for *best_cost*, *best_op_pairs*, and *chosen_op* (Lines 1-3 in Algorithm 2). Subsequently, the algorithm proceeds to evaluate all candidate operations through *brs* (Lines 4-10 in Algorithm 2).

During the evaluation, *brs* returns both a main score and a secondary operation *op'* for every operation *op* in the candidate set,

Algorithm 2: Function Select_Best_Operation

Input: *Candidate_ops*: operations to evaluate;

π : mapping function;

Output: *chosen_op*: the best operation;

```

1 best_cost  $\leftarrow -\infty$ ;
2 best_op_pairs  $\leftarrow []$ ;
3 chosen_op  $\leftarrow \emptyset$ ;
4 foreach op  $\in$  Candidate_ops do
5   main_score, op'  $\leftarrow$  brs(op,  $\pi$ );
6   if main_score > best_cost then
7     best_cost  $\leftarrow$  main_score;
8     best_op_pairs  $\leftarrow \{op, op'\}$ ;
9   else if main_score = best_cost then
10    best_op_pairs  $\leftarrow$  best_op_pairs  $\cup \{op, op'\}$ ;
11 if best_cost > 0 then
12   if |best_op_pairs| = 1 then
13    chosen_op  $\leftarrow$  best_op_pairs[0].op;
14   else
15     best_cost  $\leftarrow -\infty$ ;
16     foreach op, op'  $\in$  best_op_pairs do
17       pre_exec(op);
18        $\pi \leftarrow \pi \triangleleft op$ ;
19       pre_exec(op');
20        $\pi \leftarrow \pi \triangleleft op'$ ;
21       step3_ops  $\leftarrow$  find_ops();
22       foreach op''  $\in$  step3_ops do
23         cost3  $\leftarrow$  brs(op'');
24         if cost3 > best_cost then
25           best_cost  $\leftarrow$  cost3;
26           chosen_op  $\leftarrow$  op;
27       recover_circuit();
28 else
29   fallback();
30 return chosen_op;

```

which represents the most promising subsequent operation (Lines 4-5). The algorithm then compares the obtained score against the current best cost, updating the best solution set. Specifically, if a higher score is found, the *best_op_pairs* is reset to contain only the current optimal pair; if an equal score is encountered, the current pair is added to the set of best operation pairs (Lines 6-10).

After the evaluation phase, the algorithm enters the selection phase (Lines 11-27). It first verifies whether the *best_cost* is greater than 0, indicating the existence of beneficial operations. If only one optimal operation pair is identified, the algorithm directly selects the *op* from this pair (Lines 12-13). However, when multiple pairs have the same best score, the algorithm enters deeper lookahead evaluation (Lines

Table 2 Tie rate test for RevLib circuit set on 5 devices

Alg.	TKY	SCM	GDL	RCT	TRN	average
HLS	11.62%	20.13%	14.50%	16.33%	16.65%	15.85%
EQM	23.14%	31.69%	20.97%	23.68%	24.74%	24.84%
ILS	19.92%	30.07%	20.00%	22.80%	24.28%	23.41%

Algorithm 3: Function Adjust_Initial_Mapping

Input: LC: logical circuit;
 CG: NISQ device coupling graph;
 π : mapping function;
Output: π' : adjusted mapping;

```

1   $\pi' \leftarrow \pi$ ;
2  while True do
3       $best\_cost \leftarrow -\infty$ ;
4       $best\_op \leftarrow \emptyset$ ;
5       $ops \leftarrow find\_ops()$ ;
6      foreach  $op \in ops$  do
7           $cost \leftarrow brs(op, \pi)$ ;
8          if  $cost > best\_cost$  then
9               $best\_cost \leftarrow cost$ ;
10              $best\_op \leftarrow op$ ;
11  if  $best\_cost > 0$  then
12       $\pi' \leftarrow \pi' \triangleleft best\_op$ ;
13  else
14      break;
15 return  $\pi'$ ;
    
```

16-27). This mechanism simulates the execution of each candidate operation pair (Lines 17-20), gets the potential third-step operations, and evaluates these operations using *brs*, and ultimately selects the operation that leads to the highest score (Lines 21-26). After each lookahead evaluation, the circuit is recovered. If the best cost is less than 0, the fallback function is used to continue the process (Line 29). Through the above process, HLS can conduct deeper lookahead evaluation when encountering a tie, thereby reducing the tie rate.

Similarly, we conduct analytical experiments on the tie rate of HLS for the RevLib dataset on 5 devices (meanings of denotations of circuit sets and devices can be viewed in Section 4). As shown in Table 2, after implementing our potential-guided tie-breaking rule, the average tie rate represents a 36.2% relative reduction of EQM and 32.3% of ILS, demonstrating its improvement in decision determinism.

3.3 Iterative Initial Mapping Adjustment

Several recent research [14–18] confirmed the critical influence of the initial mapping on QMP.

The reverse search technique used by local search QMP algorithms [21, 22] was first proposed by the SABRE [13] method, which used the final mapping of a certain search for the next search in the reversed

Table 3 Coefficient of variation for RevLib circuit set on 5 devices

Alg.	TKY	SCM	GDL	RCT	TRN	average
HLS	0.018	0.008	0.008	0.005	0.005	0.009
EQM	0.028	0.008	0.008	0.007	0.008	0.012
ILS	0.033	0.011	0.012	0.010	0.010	0.015

circuit. Essentially, it is also a strategy for initial mapping. This strategy has achieved obvious improvement in solution quality, but the difference in the number of inserted auxiliary gates for the same instance running on different seeds can reach thousands on large-scale circuits due to randomness. The initial mapping for the first search is still randomly generated and has a certain exploration space.

This motivates us to adjust the randomly generated initial mapping and achieve more stable mapping quality for the same instance under different seeds. A straightforward observation is that auxiliary gate insertions started before at least one gate can be executed under the initial mapping can be ignored.

The adjustment of the initial mapping can be based on the topological characteristics of the logical circuit or focused on the first few layers. We turn our attention back to the scoring function and propose the following adjustment strategy based on it: according to the set of CNOT gates in the first few layers of the circuit, do not execute any gate, and adjust the initial mapping as much as possible until all possible operations have no positive scores, which can be denoted as:

$$\max\{brs(op, \pi)\} \leq 0 \quad (9)$$

According to Algorithm 3, the mapping optimization process operates through an iterative improvement. The algorithm begins by initializing critical variables, including setting the *best_cost* to negative infinity and *best_op* to an empty set (Lines 3-4).

The adjustment involves two main phases: evaluation and update. In the evaluation phase, the algorithm first generates the candidate operations (Line 5). It then evaluates each operation by *brs* (Lines 6-7). During this evaluation, *best_op* records the operation with the highest score (Lines 8-9).

In the update phase, the algorithm examines whether the optimal operation has a positive improvement. If so, it applies this operation to update the mapping function (Line 12); this refinement process continues iteratively. However, when no operation yields positive improvement, indicating a local optimum has been reached, the algorithm terminates the iteration and returns the adjusted mapping π' (Lines 13-15).

In local search QMP algorithms that employ a reverse search strategy, our adjustment scheme is designed specifically to improve the stability of the obtained results. To this end, we conducted an analytical experiment using the Coefficient of Variation (CV) as a metric. The formula of CV is shown in Eq. (10), where σ is the standard deviation and μ is the mean value. A lower CV value means that the physical circuit quality generated through this algorithm is more stable.

$$cv = \frac{\sigma}{\mu} \quad (10)$$

Table 4 Information of 4 circuit sets

Circuit Set	#circuits	#qubit		#CNOT	
		max	avg	max	avg
RevLib	25	20	11.2	15232	2042.8
RW	66	100	38.8	39032	6442.1
QV	60	30	17.5	1350	836.3
QUEKNO	60	133	48.5	1626	620.3

We computed the CV of HLS, EQM, and ILS for the RevLib circuit set on 5 devices, using the numbers of auxiliary gates for the same instance under seeds 0-9 (meanings of denotations of circuit sets and devices can be viewed in Section 4). As shown in Table 3, after iterated initial mapping adjustment, the coefficient of variation has a significant reduction, averaging 25.0% relative reduction of EQM and 40.0% of ILS, confirming that our iterative initial mapping adjustment can improve solution stability.

3.4 Complexity Analysis

For QMP, the inputs consist of a logical circuit $LG(Q, G)$ and a NISQ device's coupling graph $CG(V, E)$. For the overall algorithm framework, the terminal signal α and the restart signal β are tunable hyperparameters. The loop times depend on whether a better solution is found. Here, we only analyze the time complexity of a single QMP process (lines 10-18 in Algorithm 1).

In the worst case, each CNOT cannot be executed directly and is at maximum distance $|V| - 1$. Then, the loop time is $|G|$, CNOT gates needed to reduce distance to 1 or 2 by $O(|V|)$ operations. The number of candidate operations in the worst case is $O(|V|)$. For the scoring function, according to Eq. (3), ds processes CNOT gates in the first φ layers of LC . Since a layer contains at most $|Q|$ CNOT gates, and the time complexity of calculating Ψ is $|G|$, the time complexity of ds is $O(\varphi \cdot |Q| + |G|)$. $brib$ and ds evaluate the same candidate gates, with a complexity of $O(\varphi \cdot |Q|)$, so the time complexity of $bris$ is $O(2 \cdot \varphi \cdot |Q| + |G|)$. According to Eq. (7), the “max” part of brs calls bs $O(|V|)$ times, so the complexity of brs is $O(2 \cdot \varphi \cdot |Q| \cdot |V| + |G| \cdot |V|)$.

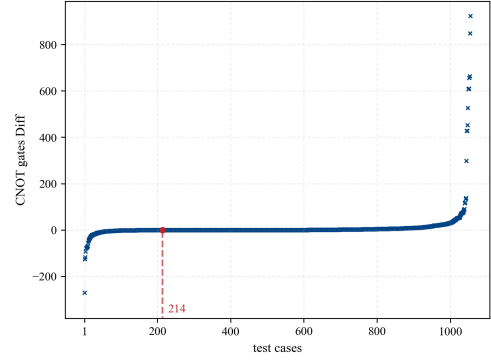
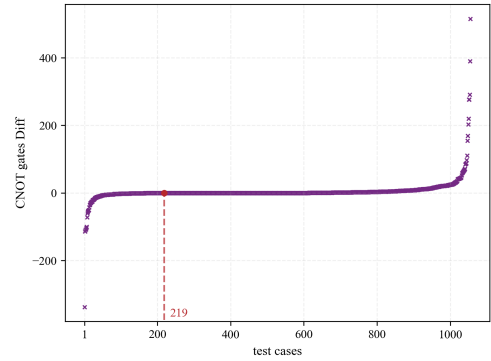
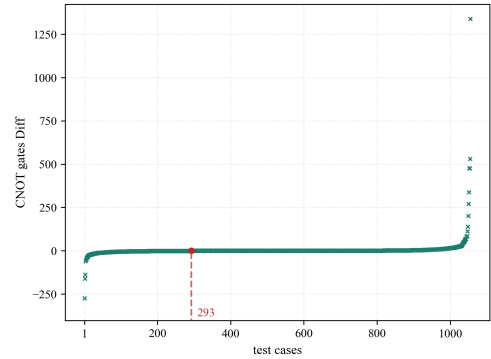
In the worst case of evaluation, where all $|V|$ candidate operations tie at depth-2 brs , $O(|V|)$ operations are evaluated by brs at depth-3. The complexity is $O(2 \cdot \varphi \cdot |Q| \cdot |V|^3 + |G| \cdot |V|^3)$. In conclusion, the time complexity of a single QMP process in HLS is $O(2 \cdot \varphi \cdot |Q| \cdot |G| \cdot |V|^4 + |G|^2 \cdot |V|^4)$.

4 Experiment

In this section, we will evaluate HLS by various experiments, and we mainly concentrate on its solving quality and effectiveness, and pay attention to the effect of three core mechanisms introduced in Section 3 at the same time.

4.1 Experimental setup

Baseline: HLS is compared against state-of-the-art QMP algorithms EQM [22] and an excellent local search QMP algorithm ILS [21], and two prevalent quantum software tools Qiskit [30] and t|ket> [31]. Qiskit is an open-source software framework developed by IBM. And we compared LightSABRE [32], denoted as LSB, which enhanced

**Fig. 6** Difference in the number of auxiliary gates between Alt1 and HLS on full test cases.**Fig. 7** Difference in the number of auxiliary gates between Alt2 and HLS on full test cases.**Fig. 8** Difference in the number of auxiliary gates between Alt3 and HLS on full test cases.

SABRE [13] in both runtime efficiency and circuit quality, and it is integrated on Qiskit. t|ket> is a powerful quantum computing compiler developed by Cambridge Quantum.

NISQ Device: We choose 5 popular NISQ devices: IBM Tokyo (see in Fig. 2b), Google Sycamore (53 qubits), IBM Guadalupe (see in Fig. 2a), IBM Rochester (53 qubits), and IBM Torino (133 qubits). We denote them as TKY, SCM, GDL, RCT, and TRN, respectively.

Benchmark: We conduct the experiment on four circuit sets: RevLib circuit set with 25 real-world logical circuits which is commonly used in previous studies [13, 16, 17, 21, 22], RW circuit set

Table 5 Detailed comparative results for RevLib circuit set on IBM Tokyo

Logical Circuit	#CNOT	HLS		EQM		ILS	
		min.(avg.)	time	min.(avg.)	time	min.(avg.)	time
co14_215	7840	2472 (2631.6)	1581.7	2640 (2736.6)	974.9	2820 (3024.9)	100.7
misex1_241	2100	387 (397.5)	221.2	408 (436.2)	78.2	459 (532.5)	14.4
decod24-v2_43	22	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
4gt13_92	30	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
sqn_258	4459	1359 (1375.8)	289.5	1458 (1485.3)	256.7	1524 (1631.4)	31.8
mod5mils_65	16	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
square_root_7	3089	591 (648.3)	1160.6	699 (777.0)	332.1	684 (829.8)	29.4
sym6_145	1701	441 (441.0)	38.5	441 (461.1)	76.1	495 (498.0)	8.7
ising_model_10	90	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
qft_13	156	60 (62.1)	126.1	60 (62.1)	45.9	66 (68.4)	3.4
qft_16	240	108 (111.6)	398.1	114 (115.5)	112.0	123 (128.7)	4.1
rd84_142	154	48 (53.7)	69.5	54 (56.1)	23.2	63 (66.3)	2.0
rd84_253	5960	2052 (2086.2)	875.3	2148 (2231.4)	347.4	2496 (2607.0)	57.4
qft_10	90	27 (27.0)	30.8	27 (27.3)	12.4	27 (28.5)	0.8
4mod5-v1_22	11	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
cycle10_2_110	2648	819 (836.4)	494.9	786 (878.1)	152.0	945 (1019.7)	24.5
z4_268	1343	384 (387.6)	256.3	411 (430.2)	103.7	456(509.1)	11.0
qft_20	380	204 (210.6)	860.9	213 (219.0)	284.0	237 (243.0)	13.3
alu-v0_27	17	3 (3.0)	0.3	3 (3.0)	0.2	3 (3.0)	0.1
sym9_193	15232	4431 (4587.0)	1120.1	5061 (5283.6)	919.1	5922 (6216.6)	153.8
adr4_197	1498	417 (452.7)	264.8	492 (500.7)	106.3	546 (569.1)	19.3
radd_250	1405	456 (465.3)	271.4	495 (526.2)	93.3	591 (600.0)	14.0
rd73_252	2319	696 (703.8)	233.5	705 (784.5)	167.8	906 (980.1)	25.6
ising_model_13	120	0 (0)	<0.1	0 (0)	<0.1	0 (0)	<0.1
ising_model_16	150	0 (0)	<0.1	0 (0)	0.1	0 (0)	<0.1

Table 6 Comparative results of QMP algorithms for RevLib circuit set on 5 devices

Device	HLS	HLS*	EQM	ILS	LSB	Qiskit	t ket>
TKY	619.2	628.8	680.6	786.0	1756.7	1878.0	1352.4
	331.7	161.3	163.4	12.9	0.1	0.1	0.7
SCM	2064.5	2068.7	2097.5	2140.0	3359.6	3398.0	2384.8
	437.6	319.3	354.6	20.6	0.2	0.1	1.0
GDL	2869.6	2870.0	2887.7	2952.7	4356.4	4394.1	3479.5
	38.3	38.2	174.6	19.0	0.2	0.1	0.7
RCT	2743.5	2749.8	2760.9	2831.7	4171.9	4241.5	3406.0
	184.0	151.2	221.0	19.4	0.2	0.1	0.8
TRN	2741.4	2748.8	2753.3	2845.6	4184.5	4246.8	3381.0
	231.4	185.9	238.6	20.5	0.2	0.1	1.0

Table 7 Comparative results of QMP algorithms for RW circuit set on 5 devices

Device	HLS	HLS*	EQM	ILS	LSB	Qiskit	t ket>
TKY	248.6	251.6	275.0	300.1	492.2	551.0	554.3
	350.8	122.0	122.1	9.0	0.1	<0.1	0.2
SCM	932.4	936.1	989.4	1093.2	1346.6	1408.2	1388.9
	1334.2	1131.1	1132.8	36.2	0.1	0.1	0.6
GDL	2095.8	2096.3	2505.2	2527.4	4869.4	4250.7	3000.8
	15.6	15.5	72.3	27.9	0.2	0.1	0.3
RCT	1568.0	1571.2	1605.9	1701.8	2386.2	2578.9	2680.7
	1173.0	907.9	916.6	47.0	0.1	0.1	0.9
TRN	3095.0	3102.1	3145.8	3435.5	4223.5	4466.3	5051.8
	1432.3	1185.4	1192.9	267.5	0.2	0.2	4.8

including 66 logical circuits from real world applications collected in [33, 34], QV circuit set composed of 60 synthetic logical circuits designed for benchmarking NISQ devices [35] and QUEKNO circuit set consisting of generated 60 circuits using QUEKNO algorithm which provides general and faithful evaluation for qubit mapping algorithms [36]. The statistical information of these circuit sets is shown in Table 4.

Hardware Environment: In this work, all experiments were conducted on a server computer equipped with dual Intel Xeon Gold 6130 processors operating at 2.10 GHz, providing a total of 64 cores, 128 threads, and 1TB of memory. The operating system version is Ubuntu 22.04.3 LTS.

Hyperparameter settings: HLS follows the local search frame introduced in Section 2.4, it has three hyperparameters: α , β , and φ . In the experiment, they are set to 500, 5, and 5, respectively.

We will conduct hyperparameter analysis, and to save space, this content is shown in the appendix. All the competitors will use the hyperparameter values recommended in their study. We use qiskit 2.4.0 with optimization_level=0, routing_method='sabre', and layout_method='sabre'. The optimization_level of qiskit can be set up to 3 to achieve better optimization results, but this process involves additional optimization. We set optimization_level=0 for fair comparison. We use pytket 2.16.0 for comparison. Except t|ket>, whose results are unrelated to seed, all cases will be tested ten times using seeds 0 to 9, and the average value will be calculated. At last, we set the cutoff time to 3600s.

4.2 Mapping quality and effectiveness

Table 5 shows the detailed comparative result of HLS, EQM, and ILS for the RevLib circuit set running on IBM Tokyo, and the best results are

Table 8 Comparative results of QMP algorithms for QV circuit set on 5 devices

Device	HLS	HLS*	EQM	ILS	LSB	Qiskit	t ket)
TKY	121.0	123.1	124.9	142.4	134.0	141.5	184.0
	565.1	180.8	180.8	6.3	<0.1	<0.1	0.2
SCM	541.6	542.8	555.1	619.2	577.5	605.4	864.0
	1389.6	1097.1	1104.9	24.9	0.1	0.1	1.0
GDL	153.3	153.9	155.9	166.3	165.1	173.6	232.1
	63.5	40.6	43.3	2.8	<0.1	<0.1	0.1
RCT	1035.7	1041.4	1053.4	1101.1	1032.3	1062.1	1536.6
	1217.1	824.4	826.3	35.3	0.1	0.1	1.4
TRN	1019.5	1023.2	1038.5	1090.4	1031.3	1087.3	1576.6
	1227.1	934.7	938.3	41.4	0.1	0.1	1.6

Table 9 Comparative results of QMP algorithms for QUEKNO circuit set on 5 devices

Device	HLS	HLS*	EQM	ILS	LSB	Qiskit	t ket)
TKY	167.4	170.7	183.2	215.4	219.2	266.2	432.1
	434.3	210.2	210.7	11.9	0.1	<0.1	0.1
SCM	240.9	243.9	263.5	324.7	323.7	382.3	577.0
	938.3	528.4	543.2	14.4	0.1	<0.1	0.2
GDL	652.5	652.8	710.9	693.5	723.2	769.0	1039.7
	45.3	42.4	94.7	9.8	0.1	<0.1	0.1
RCT	544.8	549.9	602.3	623.7	652.0	729.7	982.6
	599.4	324.7	333.4	17.1	0.1	<0.1	0.2
TRN	632.0	635.7	694.2	789.6	785.3	990.7	1433.7
	1223.2	966.1	974.6	95.9	0.1	0.1	1.2

shown in bold. For the index of auxiliary gates, HLS achieved the best average performance in all cases, and only obtained more minimum numbers of gates than EQM in “cycle10_2_110”. This demonstrates the effectiveness of HLS.

Tables 6 to 9 shows the overall performance of HLS for each device, the first row shows the average number of inserted auxiliary gates, and the second row shows the average running time.

Towards Mapping quality, HLS exceeds the competitors in all 20 conditions except the QV circuit set on RCT of LSB, and EQM’s overall performance appears the best among the baselines. As for the efficiency, from Table 5 and Table 6, HLS takes more solving time than EQM in most conditions and cases. To verify the improvement of HLS on local search QMP algorithm, and ensure a fair comparison of efficiency based on runtime, for each benchmark instance, we first determined the minimum solving time between HLS and EQM, then we recorded the number of auxiliary gates HLS obtained during this period of time, labeling this result as HLS*. The data presented in Tables 6 to 9 demonstrate that HLS* achieved superior results in all 20 cases compared to the solution obtained by EQM within the same time budget. This result provides more rigorous evidence for the efficiency improvement of HLS.

4.3 Effect of core mechanisms

In Section 3, we implemented three core mechanisms: bridge-balanced scoring function, potential-guided tie-breaking rule, and iterative initial mapping adjustment. For the scoring function, we provided illustrative examples; for the breaking rule, we analyzed and tested its effectiveness in reducing tie rates; and for the mapping adjustment, we demonstrated its contribution to improving solution stability. However, since the ultimate objectives remain solution quality and efficiency, in

this experiment section, we conduct ablation studies to evaluate the individual impact of each mechanism. The sorted results are presented in Figs. 6 to 8, where the x-axis represents each test case, including all combinations of benchmarks and devices, with a total amount of 1055. The y-axis represents the difference in the number of inserted auxiliary gates between the ablated version and HLS, and the positions of test points with a difference of 0 are marked.

For the alternative version of HLS with the bridge-balanced scoring function ablated, it used the *ps* scoring function introduced in Section 3.1; we denoted it as Alt1. From the shown result in Fig. 6, performance degradation observed in 79.7% instances demonstrates that the bridge-balanced scoring function generally provides better evaluation, thereby guiding operator selection more effectively.

The alternative version, ablated the potential-guided tie-breaking rule of HLS, is denoted as Alt2. This variant makes random selections when scores are tied, instead of performing deeper, more fine-grained evaluations. As shown in Fig. 7, performance degrades in 79.2% instances, indicating that the proposed rule can more thoroughly analyze operator benefits, reduce randomness, and lead to more accurate decision-making. The rule trades additional computation time for higher solution quality. Compared to applying deep evaluation universally and resulting in a multiplicative increase in runtime, our method remains moderate, representing a more balanced strategy.

The alternative version of HLS with iterative initial mapping adjustment ablated is denoted as Alt3. Alt3 directly uses a random initial mapping. Also, as shown in Fig. 8, 72.2% of cases obtained worse results. Although it has the least proportion of deterioration compared with the first two methods, the result still indicates that this method can achieve performance improvements in most cases, and it is reasonable, as the random initial mapping indeed accounts for a negligible proportion in the reverse search process. With significant differences observed in some test instances, it is suggested that the method can significantly enhance computational efficiency when dealing with certain cases. According to the analytical experiment in Section 3.3, the decrease in CV also reflects the method’s improvement in solution stability.

As mentioned above, all three core mechanisms of HLS contribute to improving solution quality, demonstrating their effectiveness.

5 Conclusion

In this paper, we proposed a high-performance local search method called HLS for the qubit mapping problem in the NISQ era. We introduced three core improvements: Bridge-balanced scoring function, potential-guided tie-breaking rule, and iterative initial mapping adjustment to better evaluate operations and ultimately enhance solution quality. Among them, the Bridge-balanced scoring function pays attention to the characteristics of the bridge operation and introduces a reward and penalty strategy based on distance 2, serving as a complement to existing methods. The potential-guided tie-breaking rule performs a depth-3 lookahead when ties occur. Beyond this immediate improvement, we hope this work encourages future QMP research to pay closer attention to the decision tie rate as an explicit perfor-

mance metric. Moreover, the proposed tie-breaking framework offers a methodological reference for achieving fine-grained decision-making in heuristic search under computational constraints. The iterative initial mapping adjustment is an initialization method adapted to local search. Through logically consistent and efficient optimization using the same scoring function, it works synergistically with subsequent search. We believe that, for most QMP heuristic algorithms, this simple strategy can be adapted to refine initial mapping.

We conducted an extensive evaluation of HLS on 4 circuit sets and 5 NISQ devices. Comparative experiments have demonstrated that HLS can generate significantly smaller circuits than state-of-the-art algorithms. Ablation experiments further validate the performance gains from each of the core mechanisms. The evaluation results and additional analytical experiments presented the effectiveness and robustness in reducing auxiliary gates of HLS.

The source code of HLS, all benchmarks and CGs of NISQ devices evaluated in our experiments, and evaluation results are available at <https://github.com/chuan-luo-group/HLS>.

■ Acknowledgements

This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3307500, in part by the National Natural Science Foundation of China under Grants 62522201, and 62202025, in part by the Young Elite Scientist Sponsorship Program by CAST under Grant YESS20230566, in part by Beijing Natural Science Foundation under Grant L241050, in part by CCF-Huawei Populus Grove Fund under Grant CCF-HuaweiFM2024005, and in part by the Fundamental Research Fund Project of Beihang University.

■ Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

■ Appendix

Hyperparameter analysis

The three hyperparameters of HLS are: α (the number of times the optimal solution has not been updated, corresponding to terminal signal at line 5 in Algorithm 1), β (the number of times the optimal solution has not been updated in a single search, corresponding to restart signal at line 8 in Algorithm 1), and φ (the number of evaluation layers, mentioned in Eq. (3)).

α controls the number of overall search iterations. The results are as shown in Tables 10 to 13, corresponding to values set to 50, 100, 500, and 1000. It can be observed that a larger α leads to better solutions obtained by HLS at the cost of longer runtime; β controls the number of iterations in a single search. The results are presented in Tables 14 to 17, corresponding to values set to 1, 3, 5, 7, and 9. The overall trend is similar to that of α , in most cases, a higher β imply increased runtime and improved solution quality; φ controls the depth of scoring. The results are as shown in Tables 18 to 21, corresponding to values set to 1, 3, 5, 7, and 9. It can be observed that the impact of φ varies across different devices, and its value can be flexibly chosen to achieve better results.

For different logic circuits, HLS can balance solution quality and runtime by adjusting these parameters, demonstrating strong robustness. The results for all circuit sets are available in our repository.

Table 10 Results of HLS with various value of parameter α for RevLib circuit set on 5 devices

Device	$\alpha = 50$	$\alpha = 100$	$\alpha = 500$	$\alpha = 1000$
TKY	648.9	637.8	619.2	613.6
	43.9	79.6	331.7	567.1
SCM	2091.8	2082.5	2064.5	2061.6
	58.9	108.9	437.6	742.4
GDL	2905.5	2889.4	2869.6	2860.6
	8.8	13.7	38.3	57.2
RCT	2784.7	2768.4	2743.5	2733.8
	22.6	44.1	184.0	321.6
TRN	2789.1	2771.3	2741.4	2732.5
	27.8	51.6	231.4	412.5

Table 11 Results of HLS with various value of parameter α for RW circuit set on 5 devices

Device	$\alpha = 50$	$\alpha = 100$	$\alpha = 500$	$\alpha = 1000$
TKY	265.0	254.8	248.6	248.0
	39.8	76.9	350.8	601.5
SCM	957.2	950.4	932.4	928.1
	616.4	851.7	1334.2	1502.5
GDL	2235.1	2165.2	2095.8	2080.2
	3.2	5.5	15.6	25.5
RCT	1615.1	1595.7	1568.0	1559.7
	326.5	589.3	1173.0	1376.1
TRN	3138.4	3120.7	3095.0	3093.4
	687.8	905.5	1432.3	1656.4

Table 12 Results of HLS with various value of parameter α for QV circuit set on 5 devices

Device	$\alpha = 50$	$\alpha = 100$	$\alpha = 500$	$\alpha = 1000$
TKY	125.6	124.2	121.0	119.7
	56.2	108.8	565.1	1018.1
SCM	555.0	549.6	541.5	540.4
	219.5	450.4	1389.6	1704.7
GDL	159.1	157.2	153.3	152.0
	7.4	14.4	63.5	109.8
RCT	1069.0	1057.3	1035.7	1032.1
	158.0	312.6	1217.1	1580.9
TRN	1046.5	1037.5	1019.5	1015.9
	168.1	330.1	1227.1	1619.8

Table 13 Results of HLS with various value of parameter α for QUEKNO circuit set on 5 devices

Device	$\alpha = 50$	$\alpha = 100$	$\alpha = 500$	$\alpha = 1000$
TKY	178.2	174.7	167.4	165.6
	55.1	100.2	434.3	870.2
SCM	256.1	252.6	240.9	237.9
	110.3	203.6	938.3	1439.9
GDL	665.0	660.3	652.5	649.0
	9.4	15.7	45.3	76.1
RCT	582.2	568.0	544.8	536.4
	65.6	133.1	599.4	1295.5
TRN	661.3	652.2	632.0	626.6
	605.0	696.4	1223.2	1674.9

Table 14 Results of HLS with various value of parameter β for RevLib circuit set on 5 devices

Device	$\beta = 1$	$\beta = 3$	$\beta = 5$	$\beta = 7$	$\beta = 9$
TKY	632.7	621.8	619.2	614.4	614.7
	192.4	370.7	331.7	388.5	420.3
SCM	2085.2	2072.7	2064.5	2067.5	2063.2
	292.5	514.6	437.6	507.3	583.8
GDL	2877.0	2878.9	2869.6	2870.3	2871.5
	51.7	59.6	38.3	37.5	38.5
RCT	2754.6	2746.3	2743.5	2739.8	2739.7
	103.2	234.8	184.0	207.5	200.4
TRN	2755.2	2748.0	2741.4	2734.1	2732.9
	124.9	305.9	231.4	286.2	269.2

Table 18 Results of HLS with various value of parameter φ for RevLib circuit set on 5 devices

Device	$\varphi = 1$	$\varphi = 3$	$\varphi = 5$	$\varphi = 7$	$\varphi = 9$
TKY	595.8	603.4	619.2	627.2	638.1
	416.8	293.9	331.7	501.7	472.9
SCM	2185.7	2076.9	2064.5	2039.7	2024.8
	583.7	421.5	437.6	646.3	603.6
GDL	3420.2	2910.0	2869.6	2827.1	2779.8
	99.0	40.8	38.3	62.3	59.6
RCT	3190.7	2774.6	2743.5	2694.0	2665.0
	127.7	170.9	184.0	241.1	169.1
TRN	3230.8	2780.1	2741.4	2692.3	2660.2
	151.9	195.9	231.4	234.3	403.1

Table 15 Results of HLS with various value of parameter β for RW circuit set on 5 devices

Device	$\beta = 1$	$\beta = 3$	$\beta = 5$	$\beta = 7$	$\beta = 9$
TKY	252.4	249.3	248.6	248.4	248.2
	108.0	370.7	350.8	446.4	530.0
SCM	950.1	938.7	932.4	932.5	928.7
	1020.4	1385.9	1334.2	1395.3	1433.5
GDL	2124.1	2100.6	2095.8	2083.3	2082.9
	15.2	26.4	38.3	17.9	18.0
RCT	1581.1	1570.8	1568.0	1562.1	1558.0
	764.8	1219.2	1173.0	1251.8	1306.5
TRN	3203.4	3133.2	3095.0	3107.6	3077.8
	1152.3	1445.1	1432.3	1630.2	1577.0

Table 19 Results of HLS with various value of parameter φ for RW circuit set on 5 devices

Device	$\varphi = 1$	$\varphi = 3$	$\varphi = 5$	$\varphi = 7$	$\varphi = 9$
TKY	238.0	253.1	248.6	250.4	248.4
	299.1	263.8	350.8	567.5	593.5
SCM	1137.7	967.8	932.4	933.8	951.4
	1166.5	1251.0	1334.2	1489.2	1432.7
GDL	2843.3	2606.8	2095.8	2169.6	2176.1
	57.2	25.7	15.6	31.5	29.4
RCT	2291.4	1667.3	1568.0	1590.9	1699.5
	1023.6	1145.5	1173.0	1195.1	1135.4
TRN	5258.4	3466.6	3095.0	3152.0	3738.8
	1319.9	1414.7	1432.3	1444.8	1651.8

Table 16 Results of HLS with various value of parameter β for QV circuit set on 5 devices

Device	$\beta = 1$	$\beta = 3$	$\beta = 5$	$\beta = 7$	$\beta = 9$
TKY	123.6	121.5	121.0	120.3	119.7
	344.5	573.3	565.1	950.5	907.4
SCM	552.5	545.6	541.5	541.6	538.7
	1016.3	1435.6	1389.6	1678.3	1640.7
GDL	156.4	154.5	153.3	153.5	152.8
	49.9	75.5	63.5	113.0	82.4
RCT	1058.5	1043.4	1035.7	1034.9	1032.2
	766.1	1039.4	1217.1	1566.1	1662.4
TRN	1042.6	1028.0	1019.5	1016.5	1013.1
	859.9	1025.8	1227.1	1403.5	1514.1

Table 20 Results of HLS with various value of parameter φ for QV circuit set on 5 devices

Device	$\varphi = 1$	$\varphi = 3$	$\varphi = 5$	$\varphi = 7$	$\varphi = 9$
TKY	130.7	126.4	121.0	118.9	117.8
	396.8	524.1	565.1	899.7	967.2
SCM	641.9	599.3	541.5	534.5	540.1
	849.8	1479.0	1389.6	1622.4	1527.9
GDL	173.3	161.0	153.3	154.4	157.9
	63.2	83.7	63.5	100.1	100.8
RCT	1205.4	1095.9	1035.7	1069.3	1109.4
	877.4	1493.2	1217.1	1167.8	1308.3
TRN	1192.2	1080.2	1019.5	1056.1	1099.0
	952.3	1589.3	1227.1	1196.3	1629.1

Table 17 Results of HLS with various value of parameter β for QUEKNO circuit set on 5 devices

Device	$\beta = 1$	$\beta = 3$	$\beta = 5$	$\beta = 7$	$\beta = 9$
TKY	174.2	169.7	167.4	167.6	166.4
	282.9	336.2	434.3	588.7	696.1
SCM	259.5	245.3	240.9	238.6	237.1
	656.6	769.2	938.3	1090.8	1179.1
GDL	657.6	654.9	652.5	653.2	652.2
	54.3	47.4	45.3	50.5	51.0
RCT	569.0	551.1	544.8	540.7	537.8
	393.8	455.3	599.4	779.1	904.4
TRN	716.3	656.0	632.0	627.5	626.1
	1057.2	1110.8	1223.2	1384.0	1555.4

Table 21 Results of HLS with various value of parameter φ for QUEKNO circuit set on 5 devices

Device	$\varphi = 1$	$\varphi = 3$	$\varphi = 5$	$\varphi = 7$	$\varphi = 9$
TKY	191.7	176.3	167.4	165.0	159.1
	740.9	500.8	434.3	693.6	571.3
SCM	305.0	254.8	240.9	235.4	234.5
	1027.5	1016.0	938.3	1256.8	921.8
GDL	791.1	661.6	652.5	664.8	688.3
	195.9	61.1	45.3	74.5	55.9
RCT	677.2	552.0	544.8	553.1	562.6
	697.2	658.4	599.4	625.5	666.6
TRN	776.0	650.4	632.0	658.5	665.8
	1371.4	1569.5	1223.2	1587.1	1316.7

References

- [1] Nannicini G. An introduction to quantum computing, without the physics. *SIAM Review*, 2020, 62(4): 936–981
- [2] Bauer B, Bravyi S, Motta M, Chan G K L. Quantum algorithms for quantum chemistry and quantum materials science. *Chemical reviews*, 2020, 120(22): 12685–12717
- [3] Shor P W. Algorithms for quantum computation: discrete logarithms and factoring. In: *Proceedings 35th annual symposium on foundations of computer science*. 1994, 124–134
- [4] Niu C, Rastogi P, Soman J, Tamuli K, Dixit V. Applying quantum computing to solve multicommodity network flow problem. *IEEE Transactions on Intelligent Transportation Systems*, 2025
- [5] Preskill J. Quantum computing in the nisq era and beyond. *Quantum*, 2018, 2: 79
- [6] Lau J W Z, Lim K H, Shrotriya H, Kwek L C. Nisq computing: where are we and where do we go? *AAPPS bulletin*, 2022, 32(1): 27
- [7] Siraichi M Y, Santos V F d, Collange C, Pereira F M Q. Qubit allocation. In: *Proceedings of the 2018 International Symposium on Code Generation and Optimization, CGO '18*. 2018, 113–125
- [8] Botea A, Kishimoto A, Marinescu R. On the complexity of quantum circuit compilation. In: *Proceedings of the International Symposium on Combinatorial Search*. 2018, 138–142
- [9] Jiang H, Fu J, Deng Y, Wu J. A binary integer programming-based method for qubit mapping in sparse architectures. *Acta Informatica*, 2025, 62(1): 4
- [10] Yang J, Kharkov Y A, Shi Y, Heule M J H, Dutertre B. Quantum Circuit Mapping Based on Incremental and Parallel SAT Solving. In: Chakraborty S, Jiang J H R, eds, *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*. 2024, 29:1–29:18
- [11] Molavi A, Xu A, Diges M, Pick L, Tannu S, Albarghouthi A. Qubit mapping and routing via maxsat. *Bulletin of the American Physical Society*, 2024
- [12] Zulehner A, Paler A, Wille R. An efficient methodology for mapping quantum circuits to the ibm qx architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2018, 38(7): 1226–1236
- [13] Li G, Ding Y, Xie Y. Tackling the qubit mapping problem for nisq-era quantum devices. In: *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 2019, 1001–1014
- [14] Huang Y, Zhou X, Meng F, Li S. Qubit mapping: the adaptive divide-and-conquer approach. *Quantum Information Processing*, 2025, 24(7): 202
- [15] Jiang H, Deng Y X, Xu M. Qubit mapping based on tabu search. *Journal of Computer Science and Technology*, 2024, 39(2): 421–433
- [16] Li S, Zhou X, Feng Y. Qubit mapping based on subgraph isomorphism and filtered depth-limited search. *IEEE Transactions on Computers*, 2021, 70(11): 1777–1788
- [17] Zhou X, Li S, Feng Y. Quantum circuit transformation based on simulated annealing and heuristic search. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2020, 39(12): 4683–4694
- [18] Huang C Y, Lien C H, Mak W K. Reinforcement learning and dear framework for solving the qubit mapping problem. In: *Proceedings of the 41st IEEE/ACM international conference on computer-aided design*. 2022, 1–9
- [19] Wille R, Burgholzer L. Mqt qmap: Efficient quantum circuit mapping. In: *Proceedings of the 2023 International Symposium on Physical Design*. 2023, 198–204
- [20] Itoko T, Raymond R, Imamichi T, Matsuo A. Optimization of quantum circuit mapping using gate transformation and commutation. *Integration*, 2020, 70: 43–50
- [21] Zhu P, Feng S, Guan Z. An iterated local search methodology for the qubit mapping problem. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2021, 41(8): 2587–2597
- [22] Luo C, Cao S, Guo S, Hu C. Towards effective local search for qubit mapping. *IEEE Transactions on Computers*, 2025
- [23] Hoos H H, Stützle T. Stochastic local search. In: *Handbook of approximation algorithms and metaheuristics*, 297–307. Chapman and Hall/CRC, 2018
- [24] Deutsch D. Quantum theory, the church–turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 1985, 400(1818): 97–117
- [25] Barenco A, Bennett C H, Cleve R, DiVincenzo D P, Margolus N, Shor P, Sleator T, Smolin J A, Weinfurter H. Elementary gates for quantum computation. *Physical review A*, 1995, 52(5): 3457
- [26] Oukaira A. Quantum hardware devices (qhds): Opportunities and challenges. *IEEE Access*, 2025
- [27] AbuGhanem M. Ibm quantum computers: evolution, performance, and future directions. *arXiv preprint arXiv:2410.00916*, 2024
- [28] Arute F, Arya K, Babbush R, Bacon D, Bardin J C, Barends R, Biswas R, Boixo S, Brandao F G, Buell D A, others . Quantum supremacy using a programmable superconducting processor. *Nature*, 2019, 574(7779): 505–510
- [29] Tannu S S, Qureshi M K. Not all qubits are created equal: A case for variability-aware policies for nisq-era quantum computers. In: *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 2019, 987–999
- [30] Javadi-Abhari A, Treinish M, Krsulich K, Wood C J, Lishman J, Gacon J, Martiel S, Nation P D, Bishop L S, Cross A W, others . Quantum computing with qiskit. *arXiv preprint arXiv:2405.08810*, 2024
- [31] Sivarajah S, Dilkes S, Cowtan A, Simmons W, Edgington A, Duncan R. *t|ket>*: a retargetable compiler for nisq devices. *Quantum Science and Technology*, 2020, 6(1): 014003
- [32] Zou H, Treinish M, Hartman K, Ivrii A, Lishman J. Lightsabre: A lightweight and enhanced sabre algorithm. *arXiv preprint arXiv:2409.08368*, 2024
- [33] Quetschlich N, Burgholzer L, Wille R. Mqt bench: Benchmarking software and design automation tools for quantum computing. *Quantum*, 2023, 7: 1062
- [34] Li A, Stein S, Krishnamoorthy S, Ang J. Qasmbench: A low-level quantum benchmark suite for nisq evaluation and simulation. *ACM Transactions on Quantum Computing*, 2023, 4(2): 1–26
- [35] Cross A W, Bishop L S, Sheldon S, Nation P D, Gambetta J M. Validating quantum computers using randomized model circuits. *Physical Review A*, 2019, 100(3): 032328
- [36] Li S, Zhou X, Feng Y. On constructing benchmark quantum circuits with known near-optimal transformation cost. *arXiv preprint arXiv:2301.08932*, 2023



Hongyuan Zhang received the bachelor's degree in software engineering from Beihang University, Beijing, China, in 2024. He is currently working toward the master's degree in software engineering with Beihang University, Beijing, China. His research interests include heuristic search and qubit mapping problem.



Shenghua Cao received the bachelor's degree in software engineering from China University of Geosciences, Beijing, China, in 2023. He is currently pursuing a master's degree in software engineering at Beihang University, Beijing, China. His current research interests include heuristic search and quantum computing.



Xiaoran Zhao is currently pursuing the Ph.D. degree with the School of Software, Beihang University, Beijing, China. Her research interests include deep learning, AIOps, and AI4SE.



Chuan Luo received the Ph.D. degree in computer science from Peking University, Beijing, China, in 2016. He is currently an associate professor with School of Software, Beihang University, Beijing, China. His current research interests include constraint solving and heuristic search.